

Генеративные модели

Bio



- *Карагодин Никита*
- Бакалавриат физики 2023
- Магистратура МОБС\ИИ ВШЭ 2025
- *SberDevices MLE (TTS)*

План

- Применение генеративных моделей
- Основные сведения из статистики
- Основные сведения из теории информации
- Задачи генеративного моделирования
- Авторегрессионные модели

Применение генеративных моделей

Применение генеративных моделей

Современные генеративные модели могут работать с различными модальностями



Text



Image



Sound



Video

Natural Language Processing: GPT



GPT – 1 (2018 г) заложила основу текстовых генеративных моделей:

- Требовала дообучения под конкретную задачу

GPT – 2 в 10 раз больше (1,5 млрд параметров):

- Текст стал более осмысленным
- Научилась выполнять задачи без дополнительного обучения (***zero shot***)

GPT – 3 уже до 175 млрд параметров:

- Гораздо больше обучающих данных
- Научилась справляться с задачами по примерам (***few shot***)

Natural Language Processing: GPT



GPT – 4 (2023 г) имеет $\approx 1,76$ трлн параметров:

- Увеличилось окно контекста
- Мультимодальность
- Режим размышления (*thinking mode*)

GPT – 5 (2025 г) имеет до 0,635 трлн параметров:

- Увеличилось окно контекста ($128k \rightarrow 272k$)
- Увеличилась производительность

GPT – 6 Astra (2026 г) имеет $\approx 1.5 - 2$ трлн параметров:

- Контекст 1 млн токенов
- Самостоятельное планирование шагов, управление ПК, браузером, терминалом и прочим
- Большой скачок в научных рассуждениях

Natural Language Processing: Gemini



- Разработана как конкурент *GPT – 4*
- Сразу создана как мультимодальная модель

Есть 3 версии:

- **Pro**: Топовая модель для сложных задач с режимом *Deep Think*
- **Flash**: баланс скорости и качества
- **Flash – Lite**: самая быстрая и дешевая версия

- **Gemini 1,5 (2024)**: огромное окно контекста до 1 млн токенов
- **Gemini 2,5 (2025)**: фокус на скорости (**Flash** версия) и режим мышления (**Deep Research**)
- **Gemini 3 (2025)**: общение по видео/голосом в реальном времени, управление приложениями

Natural Language Processing: Claude



- Разработан компанией **Anthropic**, основанной бывшими исследователями **OpenAI**
- Фокус – на безопасность и управляемость

Принципы конституционного ИИ:

- Модель следует набору правил, которые заложены в процесс обучения
- Безопасность встроена в архитектуру и не полагается на дообучение

Семейство моделей:

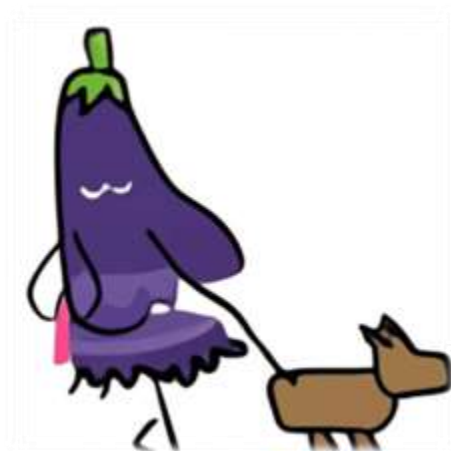
- **Fable**: флагман для самых сложных задач
- **Opus**: продвинутый уровень для сложного анализа, кода и рассуждений
- **Sonnet**: баланс скорости и интеллекта
- **Haiku**: быстрая модель для простых задач

Image Generation: DALL – E



- Модель от **OpenAI**, доступная в продуктах **OpenAI** и **Microsoft**
- Названа в честь Сальвадора Дали и робота ВАЛЛ-И

Prompt: *An illustration of an eggplant in a tutu walking a dog*



DALL- E 1



DALL- E 2

Prompt: *A hand – drawn sailboat circled by birds on the sea at sunrise*



DALL- E 2



DALL- E 3

Image Generation: Midjourney



Midjourney

- Коммерческий проект, запущенный в 2022 году
- Взаимодействие происходит через чат-бота в **Discord**

Prompt: *Pixel art scene, the eiffel tower at midnight, city lights, romantic*



V1



V2



V3



V4



V5

Audio: WaveNet

- Разработана **DeepMind (Google)**
- Используется авторегрессионный подход (*1D Conv*)
- Впервые получили речь, очень похожую на человеческую
- Очень медленная генерация



Parametric



Concatenative



WaveNet

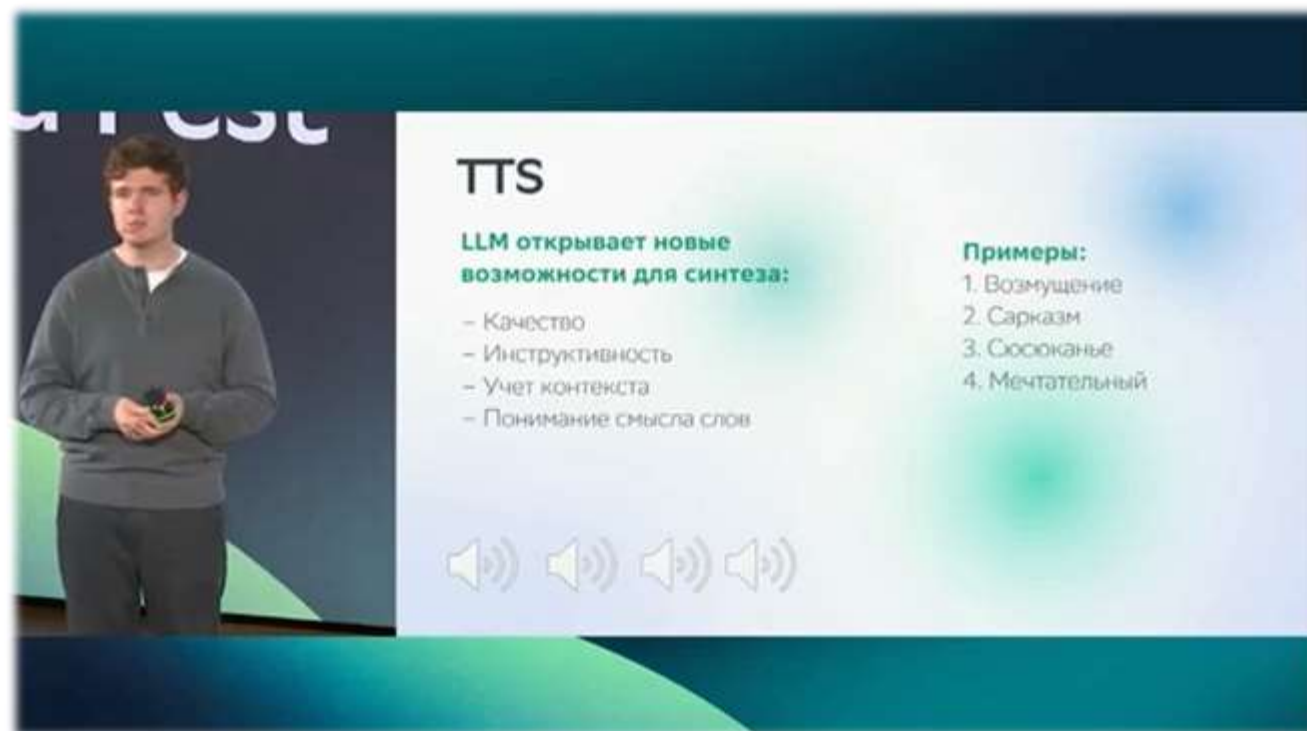
Audio: VITS

- Разработана в 2021 году
- ***End-to- End*** модель, сразу и текста получает аудио
- Использует зоопарк генеративных моделей: ***AR, Flow, VAE, GAN***



Audio: LLM Speech Synthesis

- Модели управляются богатыми текстовыми промтами, которые описывают не только что сказать, но и как это сказать
- Модели используют связку *Tokenizer* + *LLM* + *Vocoder*



TTS

LLM открывает новые возможности для синтеза:

- Качество
- Инструктивность
- Учет контекста
- Понимание смысла слов

Примеры:

1. Возмущение
2. Сарказм
3. Сосюканье
4. Мечтательный

Four speaker icons are located at the bottom of the slide.

Video: Sora

- Разработана компанией **OpenAI**
- Создает реалистичные видеоролики (без звука)
- Хорошо понимает окружающий мир и физику



Video: Veo

- Разработана в *Google DeepMind*
- Генерирует качественные видеоролики со звуком



Основные сведения из статистики

Метод максимального правдоподобия

Основная идея ММП – выбрать такие параметры модели θ , при которых наблюдаемые данные становятся наиболее вероятными

Пусть у нас есть выборка $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ из n независимых и одинаково распределенных наблюдений (*i.i.d*) из некоторого распределения $p_{\theta}(\mathbf{X})$

Правдоподобие для нашей выборки – это произведение плотностей каждого отдельного объекта:

$$f_{\theta}(\mathbf{X}) = f(\mathbf{X}|\theta) = \prod_{i=1}^n p_{\theta}(\mathbf{x}_i)$$

$f(\theta|\mathbf{X})$ – функция правдоподобия

Цель – найти такие значения θ , которые максимизируют $f(\theta|\mathbf{X})$

Метод максимального правдоподобия

Вместо функции правдоподобия обычно используют её логарифм — логарифмическую функцию правдоподобия $L_{\theta}(\mathbf{X})$:

$$L_{\theta}(\mathbf{X}) = \log f_{\theta}(\mathbf{X}) = \log \prod_{i=1}^n p_{\theta}(\mathbf{x}_i) = \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i)$$

Оценкой максимального правдоподобия (ОМП) параметров θ называют такие значение параметров, которые максимизируют функцию правдоподобия:

$$\theta_{\text{ОМП}}^* = \underset{\theta}{\operatorname{argmax}} L_{\theta}(\mathbf{X})$$

Метод максимального правдоподобия

- 1) Записать функцию правдоподобия $L_{\theta}(\mathbf{X})$
- 2) Найти градиент $L_{\theta}(\mathbf{X})$ по параметру θ
- 3) Приравнять градиент к нулю и решить уравнение относительно θ

Метод максимального правдоподобия распространён благодаря ряду свойств оценок:

- ✓ *Состоятельность*
- ✓ *Асимптотическая эффективность*
- ✓ *Асимптотическая нормальность*

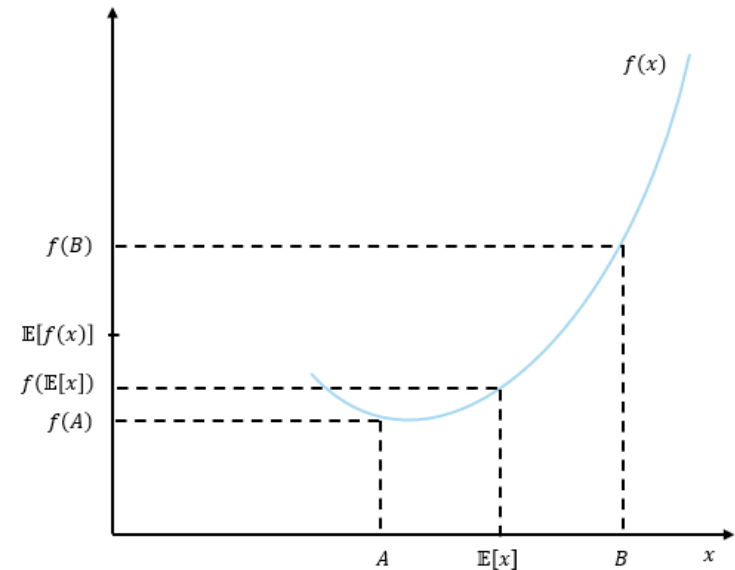
ММП был введен и подробно описан Рональдом Фишером в 1920-ые годы



Неравенство Йенсена

Для выпуклой функции f и случайной величины X выполняется следующее неравенство:

$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$$



Для выпуклой функции $f(x)$ среднее значение функции всегда больше или равно значению функции от среднего

Неравенство пригодится при вариационном выводе

Монте-Карло оценка

- Нам часто придется вычислять $\mathbb{E}_{p(\mathbf{x})}[\mathbf{f}(\mathbf{x})]$
- $p(\mathbf{x})$ многомерна и численное интегрирование невозможно

Пусть:

- $\mathbf{x} \in \mathbb{R}^m$ — случайная величина с плотностью $p(\mathbf{x})$
- $\mathbf{f}: \mathbb{R}^m \rightarrow \mathbb{R}^d$ — вектор-функция

Тогда оценка математического ожидания:

$$\mathbb{E}_{p(\mathbf{x})}[\mathbf{f}(\mathbf{x})] = \int p(\mathbf{x})\mathbf{f}(\mathbf{x})d\mathbf{x} \approx \frac{1}{n} \sum_{i=1}^n \mathbf{f}(\mathbf{x}_i), \quad \mathbf{x}_i \sim p(\mathbf{x})$$

Математическое ожидание функции $\mathbf{f}(\mathbf{x})$ можно приближённо оценить по n сэмплам из $p(\mathbf{x})$

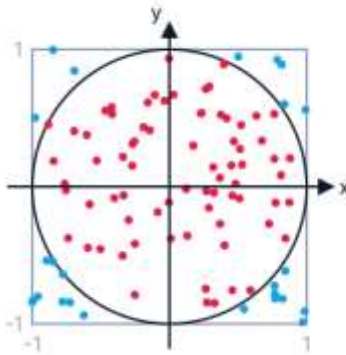
Монте-Карло оценка

- Впишем окружность с $r = 1$ в квадрат
- Равномерно разбросаем N точек внутри квадрата

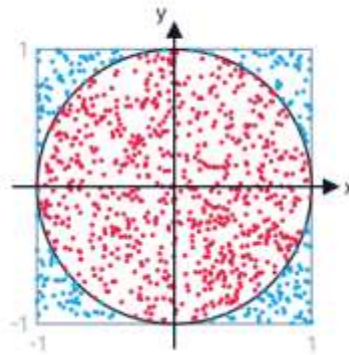
$$f(\mathbf{x}) = I[x^2 + y^2 \leq 1], \quad p(\mathbf{x}) = \frac{1}{4}$$

$$\mathbb{E}_{p(\mathbf{x})}[f(\mathbf{x})] = \int p(\mathbf{x})f(\mathbf{x})d\mathbf{x} \approx \frac{1}{N} \sum_{i=1}^N f(\mathbf{x}_i) = \frac{n}{N} \approx \frac{S_{circle}}{S_{rect}} = \frac{\pi r^2}{2r \cdot 2r} = \frac{\pi}{4} \Rightarrow \pi \approx \frac{4n}{N}$$

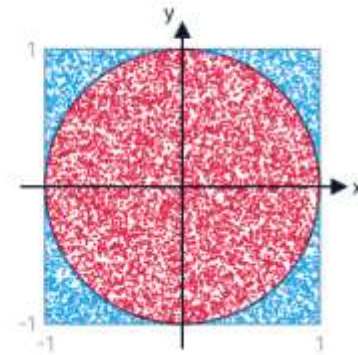
$$N = 100, n = 75 \rightarrow \pi \approx \frac{4n}{N} \approx 3.00$$



$$N = 1,000, n = 764 \rightarrow \pi \approx \frac{4n}{N} \approx 3.06$$



$$N = 10,000, n = 7822 \rightarrow \pi \approx \frac{4n}{N} \approx 3.13$$



Основные сведения из теории информации

Количество информации

Количество информации должно коррелировать со степенью нашего удивления

- Неожиданное событие $\rightarrow$ много информации
- Ожидаемое событие $\rightarrow$ мало информации
- Гарантированное событие $\rightarrow 0$ информации

Интуиция:

- Информация $h(x)$ должна быть функцией вероятности $p(x)$
- Для независимых событий информация аддитивна: $h(x, y) = h(x) + h(y)$

Этим двум свойствам удовлетворяет логарифм:

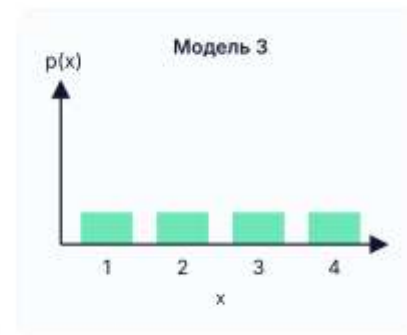
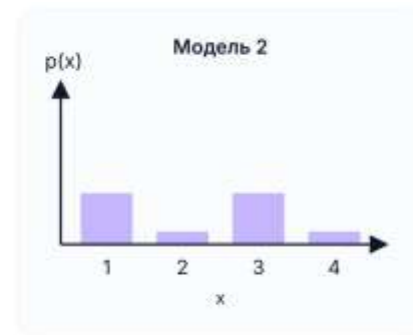
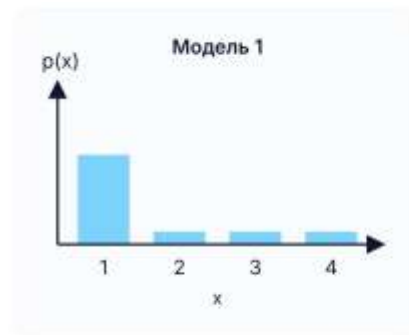
$$h(x) = -\log_2 p(x)$$

Энтропия и мера неопределенности

Среднее количество информации, которое мы получаем наблюдая за исходами случайной величины:

$$H(X) = \mathbb{E}[h(x)] = - \sum_x p(x) \log_2 p(x)$$

Эта величина называется *энтропией*



- Равномерное распределение имеет максимальную энтропию
- Неравномерное распределение (один исход сильно доминирует) имеет низкую энтропию
- Чем выше энтропия, тем менее предсказуемо распределение

Физический смысл энтропии

Энтропия – это теоретический предел для сжатия данных

По теореме Шеннона энтропия $H(X)$ – это минимальное среднее количество бит, необходимого для кодирования одного значения случайной величины

- Низкая энтропия → высокая предсказуемость → данные можно сжать эффективно
- Высокая энтропия → низкая предсказуемость → данные сжимаются плохо

Пример:

- {101, 101, 101, 101, 101}
- {010, 001, 101, 100, 111}

KL – дивергенция

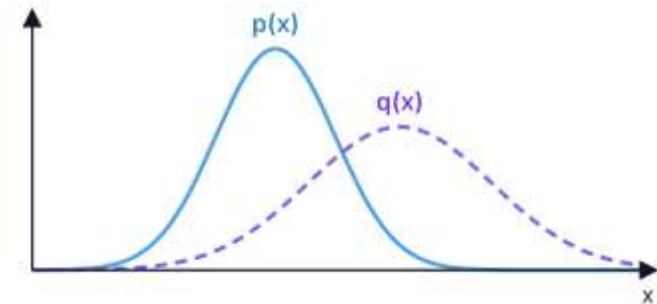
Часто нам приходится заменять истинное и неизвестное распределение $p(x)$ на более простое $q(x)$

KL – дивергенция показывает количество информации, которое мы теряем, когда аппроксимируем $p(x)$ с помощью $q(x)$:

$$\begin{aligned} D_{KL}(p|q) &= \sum_x p(x) \cdot [\log p(x) - \log q(x)] = \sum_x p(x) \cdot \log \frac{p(x)}{q(x)} \\ &= \mathbb{E}_{p(x)} \left[\log \frac{p(x)}{q(x)} \right] \end{aligned}$$

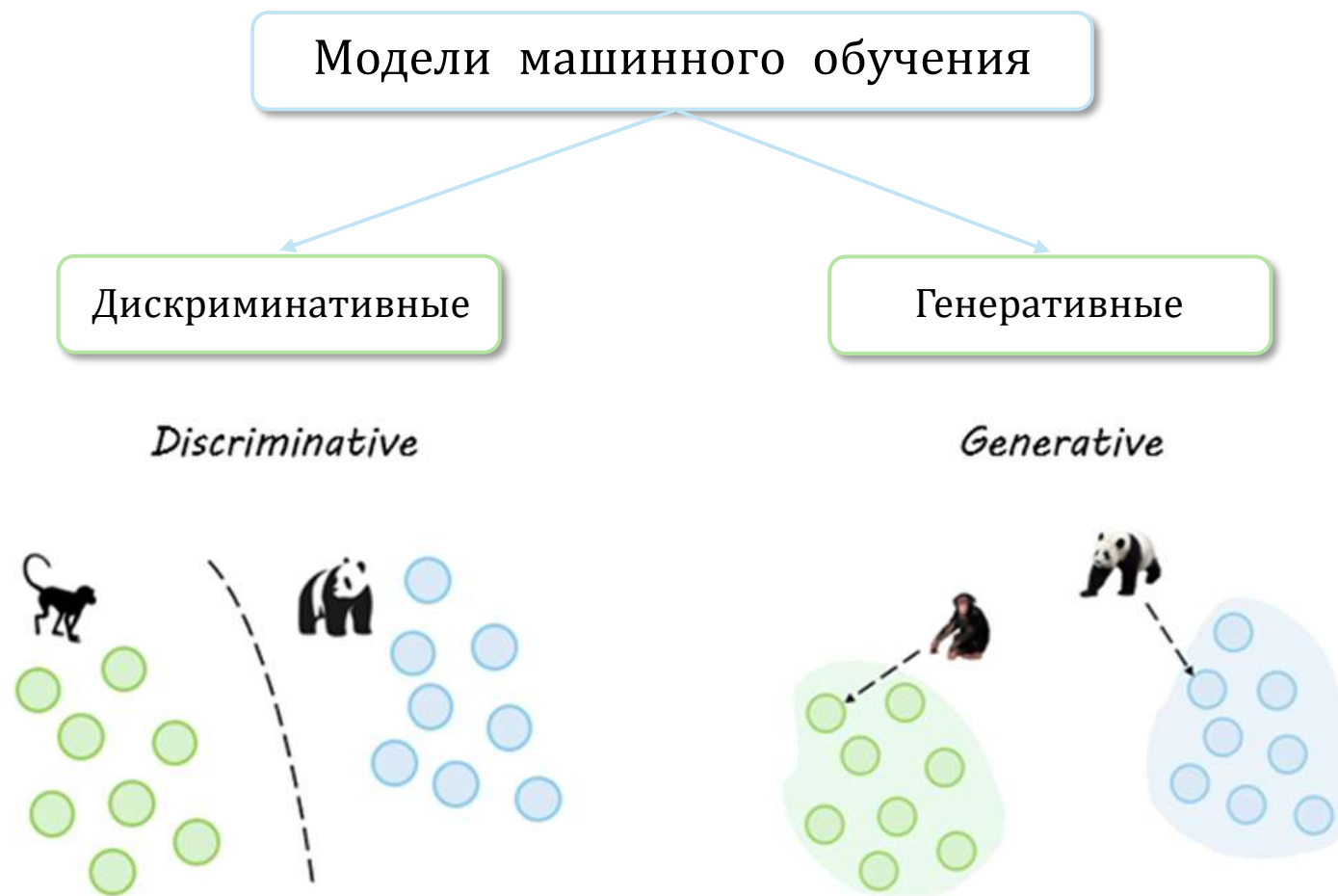
Свойства:

1. $D_{KL}(p|q) \geq 0$
2. $D_{KL}(p|q) = 0$, если $p = q$
3. Несимметричность: $D_{KL}(p|q) \neq D_{KL}(q|p)$



Задачи генеративного моделирования

Задачи генеративного моделирования

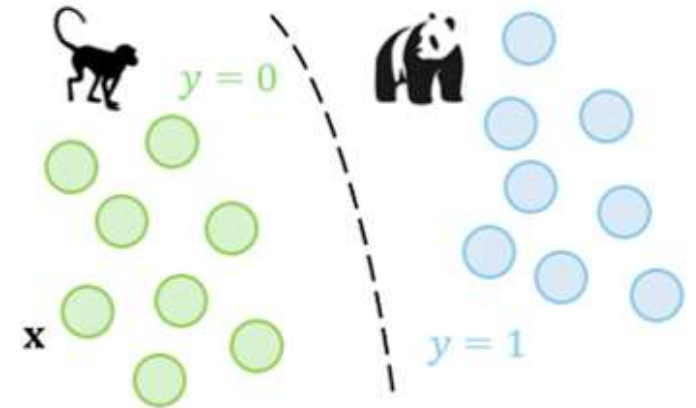


Дискриминативные модели

Предсказывают свойства объекта:

- Классификация (какой класс)
- Регрессия (число)

Цель: найти распределение $p(y|\mathbf{x})$.

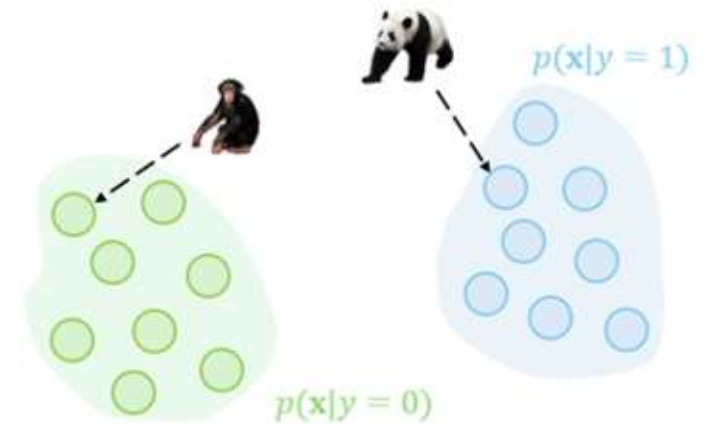


Генеративные модели

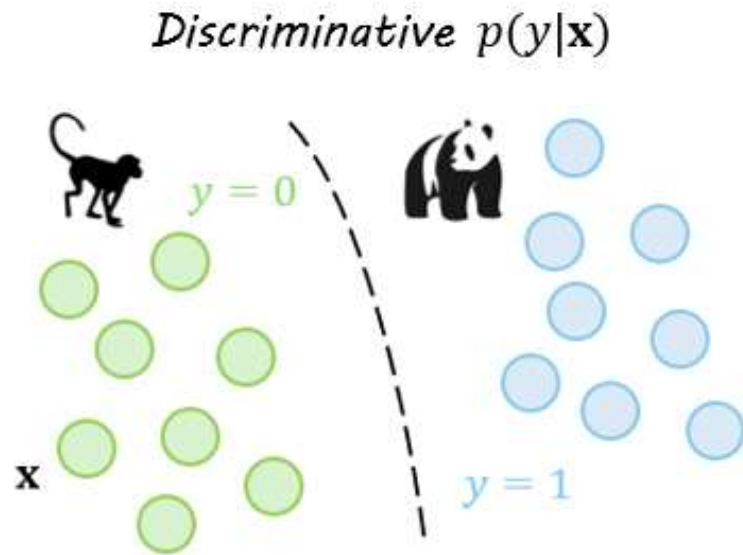
По известным свойствам объекта создают новый объект

Цель: смоделировать распределение $p(\mathbf{x})$ или $p(\mathbf{x}|\mathbf{y})$.

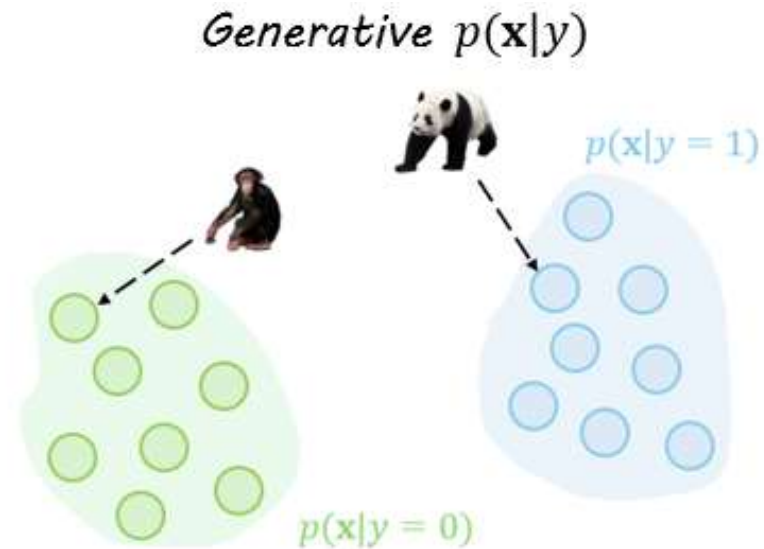
На одну метку y может приходиться бесконечное количество объектов $\mathbf{x}$.



Задачи генеративного моделирования



Определяют вероятность
характеристики y при
известном объекте x



Создают объект x по заданной
характеристике y

Ограничения дискриминативных моделей

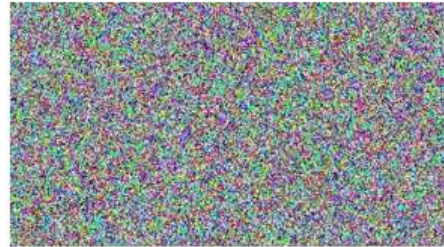
Допустим, мы обучили нейросеть, которая классифицирует изображения животных $y \in \mathcal{Y}$

$\mathcal{Y} = \{\text{кошка, собака, обезьяна}\}$



$p(y = \text{cat}|\mathbf{x}) = 0.9$
 $p(y = \text{dog}|\mathbf{x}) = 0.05$
 $p(y = \text{monkey}|\mathbf{x}) = 0.05$

+



=



$p(y = \text{cat}|\mathbf{x}) = 0.1$
 $p(y = \text{dog}|\mathbf{x}) = 0.1$
 $p(y = \text{monkey}|\mathbf{x}) = 0.8$

Нейросетям, которые выучивают $p(y|\mathbf{x})$ иногда не хватает **семантического** понимания изображений

Постановка задачи

Пусть у нас есть набор данных $\{\mathbf{x}_i\}_{i=1}^n$,

$\mathbf{x}_i \in \mathbb{R}^m$ — вектор, представляющий объект

$p_{data}(\mathbf{x})$ описывает как объекты распределены в пространстве $\mathbb{R}^m$



Цель: найти неизвестное распределение $p_{data}(\mathbf{x})$

Зная $p_{data}(\mathbf{x})$ мы сможем оценивать вероятность новых объектов и генерировать новые

Проблема высокой размерности

Объекты находятся в пространстве высокой размерности

Прямая оценка $p_{data}(\mathbf{x})$ в таких пространствах затруднена

Найти истинное распределение $p_{data}(\mathbf{x})$ почти невозможно

Будем искать аппроксимацию внутри некоторого заранее заданного *параметрического семейства распределений* $p_{\theta}(\mathbf{x})$

θ — параметры модели

Ищем θ , которые максимизируют правдоподобие:

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i)$$

Связь правдоподобия и KL -дивергенции

Наша задача — найти такие параметры θ , при которых $p_{\theta}(\mathbf{x})$ будет максимально близко к $p_{data}(\mathbf{x})$:

$$p_{\theta}(\mathbf{x}) \approx p_{data}(\mathbf{x})$$

Будем искать такие параметры θ , которые минимизируют дивергенцию между истинным и модельным распределениями:

$$\min_{\theta} D(p_{data}(\mathbf{x}) \parallel p_{\theta}(\mathbf{x}))$$

Минимизация прямой KL — дивергенции эквивалента максимизации правдоподобия

Связь правдоподобия и KL -дивергенции

Разложим формулу для прямой KL -дивергенции:

$$KL(p_{data}(\mathbf{x})|p_{\theta}(\mathbf{x})) = \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{p_{\theta}(\mathbf{x})} d\mathbf{x} = \int p_{data}(\mathbf{x}) \log p_{data}(\mathbf{x}) d\mathbf{x} - \int p_{data}(\mathbf{x}) \log p_{\theta}(\mathbf{x}) d\mathbf{x}$$

$\int p_{data}(\mathbf{x}) \log p_{data}(\mathbf{x}) d\mathbf{x}$ — энтропия истинного распределения (с минусом)

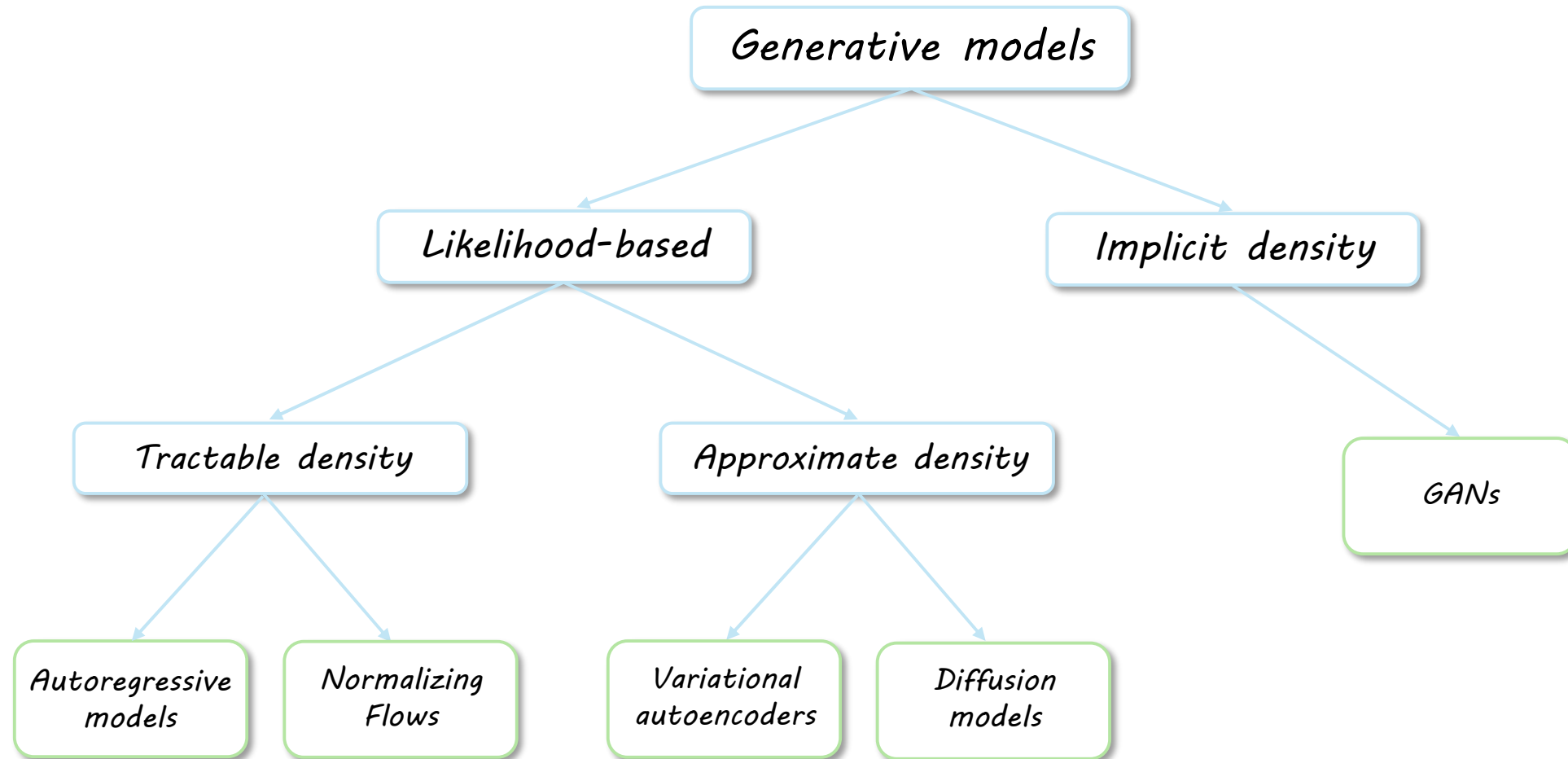
$\int p_{data}(\mathbf{x}) \log p_{\theta}(\mathbf{x}) d\mathbf{x}$ — матожидание логарифма правдоподобия по истинному распределению

Оценим интеграл:

$$KL(p_{data}(\mathbf{x})|p_{\theta}(\mathbf{x})) \approx const - \frac{1}{n} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i)$$

$$\min_{\theta} KL(p_{data}(\mathbf{x})|p_{\theta}(\mathbf{x})) \leq \max_{\theta} \frac{1}{n} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i)$$

Зоопарк генеративных моделей



Авторегрессионные модели

Обучение авторегрессионных моделей

Хотим найти параметры θ , которые максимизируют правдоподобие $p_{\theta}(\mathbf{X})$ обучающего набора данных $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$

Предполагаем, что объекты $\mathbf{x}_i$ являются независимыми, тогда:

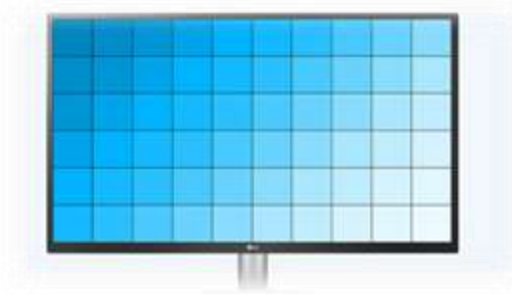
$$\theta^* = \underset{\theta}{\operatorname{argmax}} \log p_{\theta}(\mathbf{X}) = \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i)$$

Для обучения нужно быстро уметь считать $\log p_{\theta}(\mathbf{x})$ и его градиент $\nabla_{\theta} \log p_{\theta}(\mathbf{x})$

Обучение авторегрессионных моделей

Объекты в реальном мире – многомерные случайные величины

Прямое моделирование $p_{\theta}(\mathbf{x}) = p_{\theta}(x_1, x_2, \dots, x_D)$ – сложная задача из-за проклятия размерности



Разложение многомерной плотности

Разложим совместную вероятность на произведение условных:

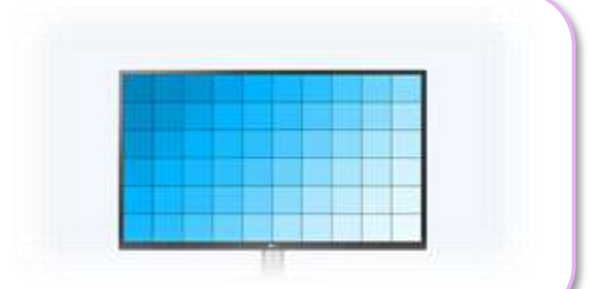
$$p_{\theta}(\mathbf{x}) = p_{\theta}(x_1, x_2, \dots, x_D) = p_{\theta}(x_1) \cdot p_{\theta}(x_2|x_1) \cdot p_{\theta}(x_3|x_1, x_2) \cdot \dots \cdot p_{\theta}(x_D|x_1, x_2, \dots, x_{D-1})$$

$$= \prod_j^D p_{\theta}(x_j | \mathbf{x}_{1:j-1})$$

$\mathbf{x}_{1:j-1}$ — вектор всех компонент объекта $\mathbf{x}$ до позиции j

Теперь задача максимизации правдоподобия принимает вид:

$$\theta^* = \underset{\theta}{argmax} \sum_{i=1}^n \sum_j^D \log p_{\theta}(x_{ij} | \mathbf{x}_{i,1:j-1})$$

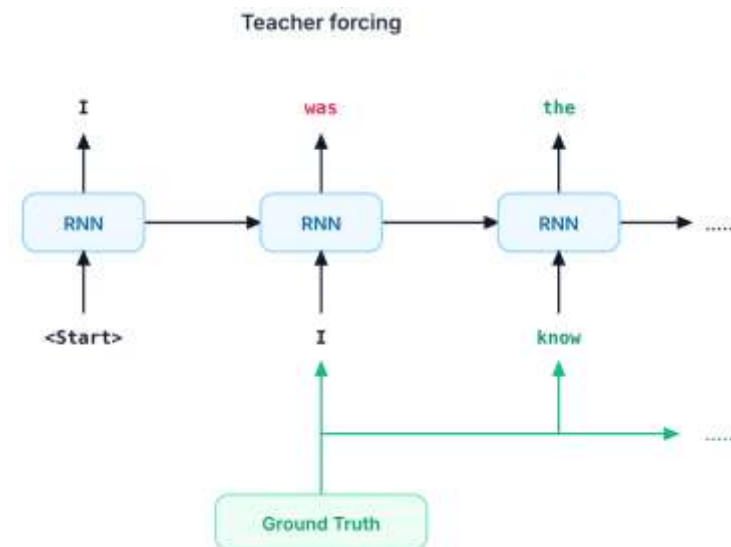
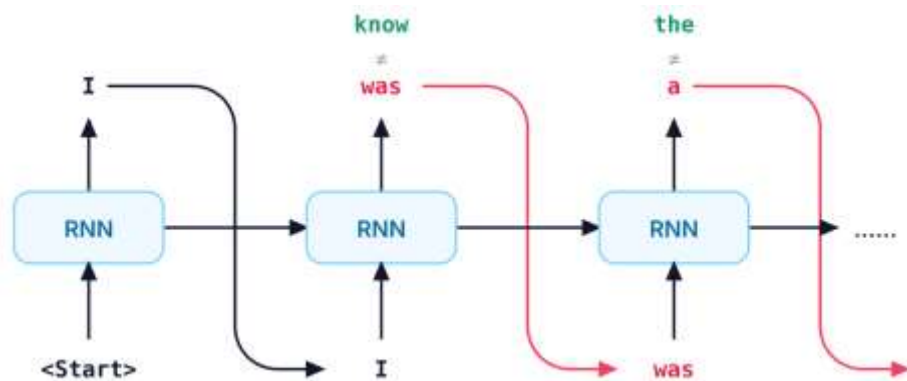


Обучение авторегрессионных моделей

В начале обучения модель часто совершает ошибки, которые с каждым шагом накапливаются

Решение:

Принудительно подавать на вход модели правильные данные (*teacher forcing*)



При генерации модель использует собственные предсказания

Авторегрессия в *RNN*

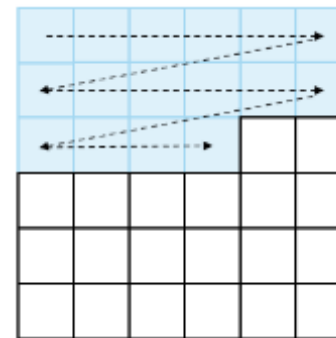
RNN по своей природе являются авторегрессионными:

$$\mathbf{h}_t = f(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

$\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}_h$ — обучаемые параметры

Звук — тоже последовательность, и *RNN* могут предсказывать следующий отсчёт

- На изображениях тоже можно задать порядок
- Обычно используют растровый порядок (*Raster Scan Order*)
- Одной из первых таких моделей была *PixelRNN*



Авторегрессия в *CNN*

- *RNN* были очень медленными и при обучении, и при генерации
- Для ускорения обучения стали использовать свёрточные сети

PixelCNN:

- Предложили использовать маскированные свертки
- На ядра свёртки накладывается маска, которая блокирует доступ к будущим пикселям

x_1					x_n
			x_i		
					x_{n^2}

1	1	1
1	0	0
0	0	0

Недостатки *PixelCNN*

Недостатки:

- Генерация все еще последовательная
- Есть слепое пятно
- Неэффективные предсказания

Последние две проблемы были решены в *Gated PixelCNN* и *PixelCNN++*



Всем

О КАК



ЛЕКЦИЯ ЗАКОНЧИЛАСЬ